

Doubly Linked List – Creation of Empty List –Space Complexity and Algorithm.

Program :

```
#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
}Node;

Node *head;

void createEmptyList()
{
    head = nullptr;
}

int main()
{
    createEmptyList();
    return 0;
}
```

Space Complexity

The space complexity of a program refers to the amount of memory space required by the program as a function of the input size. In this particular program, the space complexity is determined by the following factors:

1. **Node Structure:** The DLL structure is defined to hold an integer (`int data`), a pointer to the next node (`DLL * next`), a pointer to the previous node (`DLL * prev`).
The space required for each node is constant, i. e., $O(1)$.
2. **Head Pointer:** The head pointer is a global variable that points to the start of the linked list. It requires a constant amount of space, i. e., $O(1)$.
3. **Function Call:** The `createEmptyList` function takes a pointer to a pointer as an argument. The space required for this function call is also constant, i. e., $O(1)$.

Since the program only initializes an empty list and does not create any nodes dynamically, the overall space complexity of this program is $O(1)$.

1. Input Space Complexity: $O(1)$

Input space is the memory required to store the parameters passed to a function.

- **Parameter-Only & Full-Data Definition:** The `createEmptyList()` function takes no parameters. Since there is no input, the space required for it is zero, which is a constant amount.

Therefore, under both definitions, the input space complexity is $O(1)$.

2. Auxiliary Space Complexity: $O(1)$

Auxiliary space is the extra memory an algorithm uses during its execution.

1. **No New Variables:** The `createEmptyList` function does not declare any of its own local variables.
2. **Single Operation:** It performs a single assignment (`head = nullptr;`) on a global variable. This operation does not consume any additional memory.

Because the function uses no extra memory to perform its task, its auxiliary space complexity is constant.

Therefore, the auxiliary space complexity is $O(1)$.

Algorithm of Creation of Empty List

Algorithm of Creation of Empty Doubly Linked List

Step 1: DEFINE STRUCTURE

- Define a structure DLL with three members:
 - An integer data to store the value of the node.
 - A pointer next of type DLL* to point to the next node in the list.
 - A pointer prev of type DLL* to point to the previous node in the list.

Step 2: Type Alias

- Use typedef to create an alias Node for struct DLL to simplify the code.

Step 3: DECLARE GLOBAL POINTER

- Declare a global pointer head of type Node* to keep track of the head of the list.

Step 4: CREATE FUNCTION

- Define a function createEmptyList that takes no arguments.
- SET HEAD := NULLPTR

Step 5: Call Function

- Call Function → createEmptyList.
-
